

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

TRABAJO:
Laboratorio 8

Estudiante:
Sahid Eliel Morales Filos (8-1133-1580)
Reynaldo Murgas (8-1019-1644)

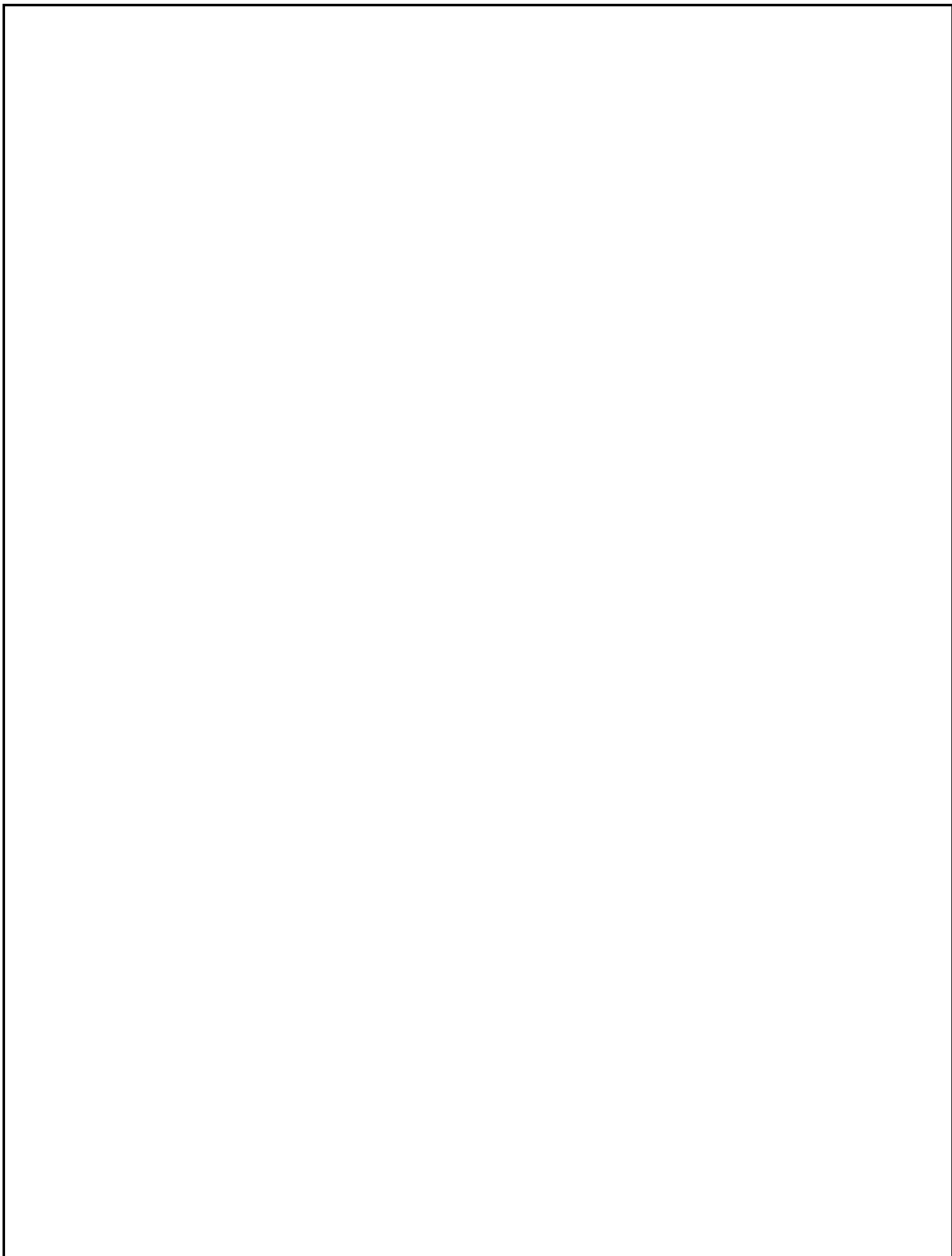
Curso:
Ingeniería de Software IV

Profesor:
Carmen Ortega

Grupo:
9GS141

Semestre:
I

4/62026



Asignación 1: Evaluación Cuantitativa de Requisitos (25 Puntos)

- Formule y calcule de manera matemática el Índice de Completitud de Requisitos (Q_3) aplicable a la plataforma HealthConnect. Es mandatorio desglosar la fórmula empleada, sustituir los valores numéricos del escenario y reflejar explícitamente el procedimiento aritmético seguido.
- Con base en el resultado obtenido (un rango de 0 a 1), provea una interpretación técnica rigurosa sobre el estado real de la especificación de requisitos. Analice cómo la falta de completitud estructural en la etapa inicial funcionó como un vector de propagación de fallas que detonó la crisis en el ambiente operativo.

Desarrollo:

1.1. Datos del caso

El documento de especificación original de HealthConnect contenía un alcance cerrado de 80 requisitos funcionales. Según la auditoría, solo 55 requisitos estaban validados, estructurados y completos. Los 25 requisitos restantes se encontraban marcados como TBD (por definir) o presentaban descripciones ambiguas, por ejemplo: "El sistema web debe procesar todas las solicitudes médicas internas de forma extremadamente veloz".

1.2. Fórmula aplicada

Para calcular el Índice de Completitud de Requisitos se utiliza la métrica Q_3 :

$$Q_3 = Nu / (Nu + Ni)$$

Donde:

Nu = número de requisitos únicos, validados y no ambiguos

Ni = número de requisitos incompletos o pendientes de validación

1.3. Sustitución de valores y cálculo

$$Nu = 55$$

$$Ni = 25$$

$$Q_3 = 55 / (55 + 25)$$

$$Q_3 = 55 / 80$$

$$Q_3 = 0.6875 \times 100$$

$$Q_3 = 68.75 \%$$

Asignación 2: Diagnóstico de la Arquitectura y Diseño Lógico (25 Puntos)

- Identifique y defina formalmente las dos métricas de diseño orientado a objetos (catálogo clásico de Chidamber y Kemerer) que están siendo severamente infringidas por la clase crítica MedicalRecordController. Indique razonadamente, con base en los datos provistos en el informe de auditoría, si los valores de estas métricas se sitúan en niveles excesivamente altos o bajos.
- Describa detalladamente el impacto estructural negativo que estos valores ejercen sobre el producto de software, haciendo énfasis en conceptos críticos de ingeniería como el 'efecto dominó' ante solicitudes de cambio, la fragilidad del código y la degradación del Principio de Responsabilidad Única (SRP).

Desarrollo:

2.1. Métricas afectadas en MedicalRecordController

La clase crítica MedicalRecordController presenta dos problemas severos de diseño orientado a objetos. Primero, interactúa de manera directa con 28 clases periféricas del ecosistema. Segundo, posee 15 métodos activos, pero ninguno comparte las mismas variables de instancia. Estos datos evidencian infracciones graves en dos métricas clásicas de Chidamber y Kemerer: CBO y LCOM.

Métrica	Definición formal	Dato del caso	Diagnóstico
CBO (Coupling Between Object Classes)	Mide la cantidad de clases externas con las que una clase se encuentra directamente vinculada.	MedicalRecordController interactúa con 28 clases periféricas.	CBO excesivamente alto. La clase posee demasiadas dependencias directas.
LCOM (Lack of Cohesion of Methods)	Mide la falta de relación entre los métodos de una clase con respecto a los atributos o variables de instancia que comparten.	15 métodos activos no comparten variables de instancia.	LCOM críticamente alto y cohesión muy baja. La clase concentra responsabilidades no relacionadas.

2.2. Diagnóstico del acoplamiento CBO

El CBO de MedicalRecordController es 28, porque la clase está vinculada directamente con 28 clases periféricas. Este valor es excesivamente alto para una clase crítica del backend. Un CBO elevado indica que la clase depende de muchos componentes externos y, por tanto, cualquier cambio en esos componentes puede impactar de forma inesperada el comportamiento de la clase central.

Para poder definir o traducir de manera entendible medida del CBO podemos decir lo siguiente:

- 0 - 5 = Bajo acoplamiento**
- 6-10 = Acoplamiento aceptable**
- 11-20 = acoplamiento alto, requiere revisión**
- >20 = acoplamiento crítico, requiere refactorización**

El efecto negativo más importante es el efecto dominó. Si se modifica una clase relacionada con pacientes, expedientes, citas, teleconsultas, validaciones o transacciones, existe una probabilidad alta de que MedicalRecordController también deba modificarse. Esto aumenta la fragilidad del sistema, dificulta las pruebas unitarias y genera mayor riesgo de defectos en producción.

2.3. Diagnóstico de la falta de cohesión LCOM

El LCOM también se encuentra en un nivel crítico. Si se parte de 15 métodos activos, el número total de pares de métodos posibles es:

Como el informe indica que ningún método comparte variables de instancia, los 105 pares de métodos no tienen relación interna. En una interpretación clásica de LCOM, esto implica un valor aproximado de 105, lo que representa una falta de cohesión severa.

Podemos explicarlo mediante la siguiente ecuación:

$$LCOM = P - Q$$

P = pares de métodos que no comparten variables

Q = pares de métodos que si comparten variables

$$Total\ de\ pares = \frac{15(15 - 1)}{2}$$
$$Total\ de\ pares = \frac{15 \times 14}{2}$$

$$Total\ de\ pares = 105$$

P= 105

Q= 0

LCOM = 105 – 0 =105

Y podemos normalizarlo a

$$\begin{aligned} LCOM_{normalizado} &= \frac{LCOM}{Total\ de\ pares} \\ LCOM_{normalizado} &= \frac{105}{105} \\ LCOM_{normalizado} &= 1.0 \end{aligned}$$

Eso equivale a:

100% de falta de cohesión

Entonces, la cohesión real sería:

$$\begin{aligned} Cohesión &= 1 - LCOM_{normalizado} \\ Cohesión &= 1 - 1.0 \\ Cohesión &= 0 \end{aligned}$$

Umbral recomendado:

LCOM normalizado Interpretación

0.00 - 0.30	Cohesión aceptable
0.31 - 0.50	Advertencia, revisar diseño
0.51 - 0.80	Problema serio de cohesión
0.81 - 1.00	Problema crítico, requiere refactorización

Este resultado demuestra que MedicalRecordController viola el Principio de Responsabilidad Única (SRP). En vez de cumplir una responsabilidad clara, la clase parece concentrar múltiples tareas diferentes dentro de un mismo controlador. Esto degrada la mantenibilidad, aumenta la complejidad de lectura y dificulta que un desarrollador pueda modificar una parte del sistema sin afectar otras.

2.4. Impacto estructural sobre el producto software

Los valores de CBO y LCOM muestran que MedicalRecordController se comporta como una clase central excesiva o clase "Dios". Este tipo de diseño genera una arquitectura frágil, difícil de probar, difícil de modificar y propensa a fallos secundarios. En un sistema médico, esta situación es especialmente delicada porque cualquier error puede afectar procesos sensibles como expedientes clínicos, citas y teleconsultas.

La solución técnica recomendada es dividir la clase en componentes especializados, por ejemplo: MedicalRecordService, AppointmentService, TeleconsultationService, ValidationService y ClinicalAnalyticsService. Cada componente debe tener responsabilidades claras, menor acoplamiento, mayor cohesión y pruebas unitarias independientes.

Asignación 3: Análisis Estático de Complejidad Algorítmica (25 Puntos)

- Calcule formalmente la Complejidad Ciclomática de McCabe ($V(G)$) correspondiente al método crítico evaluarHistorialPaciente(). Utilice la ecuación estándar del modelado de grafos ($V(G) = E - N + 2P$), asumiendo un único componente conexo en la arquitectura ($P = 1$). Muestre detalladamente el desglose aritmético.
- Contraste el valor numérico resultante con los rangos internacionales de riesgo estipulados por el Software Engineering Institute (SEI). Clasifique formalmente el nivel de riesgo técnico del método y explique con rigor analítico qué implicaciones directas posee este número para el equipo de Aseguramiento de Calidad (QA) con respecto al diseño de pruebas unitarias y la mantenibilidad del sistema a mediano plazo.

Desarrollo:

3.1. Datos del método crítico

El método evaluarHistorialPaciente() fue identificado como una rutina crítica del backend. El análisis estático reportó múltiples estructuras condicionales anidadas, bucles while y selectores switch redundantes. El grafo de flujo del método contiene 62 aristas, 14 nodos y se asume un único componente conexo.

3.2. Fórmula de McCabe

$$V(G) = E - N + 2P$$

Donde:

E = número de aristas del grafo de flujo

N = número de nodos del grafo

P = número de componentes conexos

3.3. Sustitución de valores y cálculo

$$E = 62$$

$$N = 14$$

$$P = 1$$

$$V(G) = 62 - 14 + 2(1)$$

$$V(G) = 62 - 14 + 2$$

$$V(G) = 50$$

3.4. Clasificación según los rangos de riesgo del SEI

Rango V(G)	Interpretación	Nivel de riesgo
1 - 10	Código simple	Bajo
11 - 20	Complejidad moderada	Controlado
21 - 50	Alta complejidad	Alto
Mayor a 50	Código inestable o espagueti	Crítico

El resultado obtenido para evaluar `HistorialPaciente()` es $V(G) = 50$. Según los rangos del SEI, este método se clasifica como código de alta complejidad y alto riesgo. Además, se encuentra en el límite superior del rango, justo antes de pasar a la categoría de código inestable o espagueti.

3.5. Implicaciones para QA y mantenibilidad

La complejidad ciclomática indica la cantidad de caminos independientes que deben probarse para alcanzar una cobertura lógica adecuada. Por tanto, un valor de 50 implica que el equipo de QA tendría que diseñar aproximadamente 50 casos de prueba independientes para cubrir la lógica principal del método. Esto vuelve el aseguramiento de calidad más lento, costoso y propenso a omisiones.

Desde la mantenibilidad, un método con esta complejidad es difícil de entender, modificar y depurar. Cada nuevo cambio aumenta el riesgo de introducir errores colaterales. La recomendación técnica es refactorizar el método en funciones más pequeñas, eliminar condicionales redundantes, aplicar validaciones tempranas, usar estrategias de diseño y separar las reglas clínicas en servicios o clases especializadas.

Asignación 4: Marco de Gobernanza Informática y Dashboard Estratégico (25 Puntos)

- El equipo de desarrollo remanente sugiere a la gerencia que la vía rápida para solucionar la crisis actual consiste simplemente en 'programar código de manera más acelerada para mitigar los bugs reportados de forma inmediata'. Basándose estrictamente en la Ley de Goodhart y en los Principios Generales de la Medición de Software analizados en la asignatura, elabore un argumento crítico que refute esta postura y justifique por qué un enfoque enfocado en metas punitivas empeorará la arquitectura técnica de la plataforma.
- Diseñe un Tablero de Control Automatizado (Dashboard de Calidad) estructurado mediante una tabla formal, especificando las métricas de control que HealthConnect debe implementar obligatoriamente en su pipeline de Integración Continua (CI/CD) para impedir futuros despliegues fallidos. La estructura de la tabla debe contener los siguientes campos exactos:
 - Nombre Técnico del Indicador Estratégico.
 - Métricas directas o indirectas que lo computan.
 - Umbral de Aceptación (Condición de Semáforo Verde - Paso Libre).
 - Umbral de Alerta y Rechazo Crítico (Condición de Semáforo Rojo - Bloqueo de Despliegue).

Desarrollo:

4.1. Refutación de la postura de "programar más rápido"

La propuesta de solucionar la crisis simplemente programando más rápido es incorrecta desde la perspectiva de gobernanza de software. HealthConnect no colapsó por falta de velocidad, sino por ausencia de medición, revisiones técnicas, requisitos completos, controles de arquitectura, pruebas y validaciones de seguridad antes del despliegue.

La Ley de Goodhart establece que cuando una medida se convierte en un objetivo rígido, deja de ser una buena medida. Si la gerencia presiona al equipo únicamente con metas como "corregir más bugs por día" o "liberar más rápido", los desarrolladores pueden enfocarse en producir parches superficiales para cumplir la métrica, pero sin resolver las causas estructurales del problema. Esto puede aumentar la deuda técnica, el acoplamiento, la complejidad y las vulnerabilidades.

Los principios generales de medición de software indican que las métricas deben tener un objetivo claro, ser simples de recopilar, ser válidas y usarse en una cultura libre de culpa. Por ello, HealthConnect debe implementar un enfoque de mejora continua basado en indicadores de calidad automatizados, no en castigos ni metas punitivas. El propósito de las métricas debe ser diagnosticar, prevenir y mejorar, no presionar al equipo para producir código sin control.

4.2. Dashboard de Calidad propuesto para CI/CD

El siguiente tablero de control automatizado debe integrarse en el pipeline CI/CD para impedir que módulos incompletos, inseguros, lentos o excesivamente complejos lleguen nuevamente a producción.

Nombre Técnico del Indicador Estratégico	Métricas directas o indirectas que lo computan	Umbral de Aceptación (Condición de Semáforo Verde - Paso Libre)	Umbral de Alerta y Rechazo Crítico (Condición de Semáforo Rojo - Bloqueo de Despliegue)
Índice de Completitud de Requisitos	Q3 = requisitos completos / total de requisitos.	Q3 $\geq$ 0.90 en módulos críticos.	Q3 < 0.80 bloquea el despliegue hasta completar o validar requisitos.
Control de Ambigüedad de Requisitos	Cantidad y porcentaje de requisitos TBD, ambiguos o sin criterio de aceptación.	0 requisitos TBD en funcionalidades críticas y menos de 5 % de ambigüedad total.	Más de 5 % de requisitos ambiguos o cualquier TBD crítico bloquea el release.
Control de Complejidad Ciclomática	Complejidad de McCabe V(G) por método.	V(G) $\leq$ 10 en el 95 % de métodos y ningún método crítico por encima de 15.	Cualquier método crítico con V(G) > 20 bloquea el despliegue y exige refactorización.
Control de Acoplamiento Arquitectónico	CBO por clase crítica.	CBO $\leq$ 10 en clases críticas.	CBO > 20 en una clase crítica bloquea aprobación técnica.
Control de Cohesión de Clases	LCOM por clase y revisión de responsabilidades.	LCOM bajo o evidencia de métodos relacionados con los mismos atributos.	LCOM alto en clase crítica bloquea despliegue y exige separación de responsabilidades.
Control de Seguridad del Código	Vulnerabilidades detectadas por SAST, análisis de dependencias y pruebas de inyección.	0 vulnerabilidades críticas o altas.	Cualquier vulnerabilidad crítica o alta bloquea despliegue.
Control de Cobertura de Pruebas	Porcentaje de cobertura unitaria, integración y pruebas de regresión.	Cobertura $\geq$ 80 % en módulos críticos.	Cobertura < 70 % en módulos críticos bloquea despliegue.

Nombre Técnico del Indicador Estratégico	Métricas directas o indirectas que lo computan	Umbral de Aceptación (Condición de Semáforo Verde - Paso Libre)	Umbral de Alerta y Rechazo Crítico (Condición de Semáforo Rojo - Bloqueo de Despliegue)
Control de Rendimiento Operativo	Tiempo promedio y percentil 95 de respuesta en pruebas de carga.	Respuesta $\leq$ 2 segundos en operaciones críticas bajo carga esperada.	Respuesta $>$ 5 segundos o degradación severa bloquea despliegue.
Control de Memoria	Fugas de memoria y consumo máximo durante pruebas automatizadas.	0 fugas críticas y consumo dentro del límite establecido.	Cualquier fuga crítica o crecimiento continuo de memoria bloquea release.
Densidad de Defectos	Defectos confirmados / KLOC.	$\leq$ 2 defectos por KLOC.	$>$ 4 defectos por KLOC bloquea despliegue y activa revisión de calidad.
Control de Revisión Técnica	Porcentaje de cambios con peer review aprobado.	100 % de pull requests críticos con revisión aprobada.	Cualquier cambio crítico sin revisión bloquea integración a producción.

4.3. Acciones correctivas recomendadas

Completar y validar los requisitos antes de desarrollar nuevos módulos, eliminando términos ambiguos como "rápido" o "extremadamente veloz" y sustituyéndolos por criterios medibles.

Restablecer revisiones entre pares obligatorias para todo cambio crítico antes de integrarlo al repositorio principal.

Refactorizar MedicalRecordController para reducir acoplamiento, mejorar cohesión y cumplir el Principio de Responsabilidad Única.

Dividir evaluarHistorialPaciente() en métodos pequeños, probables y con reglas clínicas separadas por responsabilidad.

Incorporar herramientas automatizadas de análisis estático, pruebas unitarias, pruebas de rendimiento y análisis de seguridad dentro del pipeline CI/CD.

Usar el dashboard como mecanismo de prevención y mejora continua, no como instrumento punitivo contra el equipo de desarrollo.

Conclusión

El uso de estas herramientas y métricas es de gran ayuda para la medición y poder ver o entender de manera que podamos ubicar o comparar matemáticamente el cómo está de bien o mal el proyecto o producto, de forma que podamos reflejar o comunicar mejor las mejoras o fallas y sea entendible para otras personas. (Sahid Morales)

Las métricas evidenciaron problemas estructurales que pudieron haberse prevenido mediante controles de calidad y monitoreo continuo, reafirmando la necesidad de una gestión basada en datos para desarrollar software más confiable y mantenible. (Reynaldo Murgas)